Claude Code — Course-Ready Blueprint & Auditor Validation

Executive overview

This document is a finalized, course-ready blueprint for a professional developer training program teaching Claude Code — a terminal-first, agentic coding assistant experience. It validates the researched course structure from installation through advanced agent orchestration, highlights missing or ambiguous items to resolve before publishing, and provides precise, production-aligned guidance for delivering the course (lecture/demos, labs, assessments).

Learning outcomes (high level)

- Install, authenticate, and configure Claude Code CLI and VS Code extension safely.
- Run interactive project sessions and author CLAUDE.md to shape assistant behavior.
- Execute common CLI commands safely (open, edit, run, search, commit) and integrate with Git/CI.
- Apply multi-file reasoning, debugging loops, and test-driven fixes.
- Build agentic workflows, custom tools, and CI integrations while enforcing enterprise guardrails.
- Evaluate and compare Claude Code to other AI developer tools and deploy production workflows.

Auditor summary (structural validation)

- Structural soundness: The course follows a clear progression: Setup → Core Workflows → Intermediate capabilities (session/model/prompting/refactors) → Advanced (agents, tools, governance, scaling, capstone). Units logically build on previous ones.
- Completeness: Core topics covered end-to-end (install → CLAUDE.md → sessions → commands → models → agent flows → governance). Minor gaps: explicit .clauderc schema examples, precise current CLI flags for every command (must be validated against live CLI), SSO/enterprise setup steps (depends on Anthropic offering).
- Practical readiness: Units include demonstrations and realistic projects; lab cadence and capstone are defined.
- Risks & assumptions: Course assumes availability of official Claude Code CLI and extension; specific command flags/syntax should be verified at course publication time.

Level distribution summary

- Beginner (Units 1–8): Install, auth, first sessions, CLAUDE.md, core commands, VS Code integration, simple tests, workspace hygiene.
- Intermediate (Units 9–18): CLI config, sessions/workspaces, model selection/prompting, scripting, debugging, multi-file refactors, Git workflows.
- Advanced (Units 19–36): Agent orchestration, custom tools, sandboxing, scaling to large repos, memory & performance, enterprise governance, CI/CD integration, capstone.

Structured complete unit list (validated) Below is the finalized unit list. Each unit includes: title, level, learning objective, and exact demonstration (actionable steps). Where command

syntax is uncertain, the "Action" notes request verifying flags against the installed CLI version.

Beginner (Units 1–8)

1. Unit 1 — Install Claude Code CLI and prerequisites
    - Level: Beginner
    - Objective: Install the CLI and verify environment readiness
    - Demonstration:
        - Verify prerequisites (Node/npm optional if distributed via npm; Git installed)
        - Install CLI (examples):
            - npm i -g @anthropic/claude-code
            - or Homebrew / apt package (if available)
        - Run: claude --version
        - Action: Validate actual package name and installer on publication
2. Unit 2 — Authentication and account configuration
    - Level: Beginner
    - Objective: Authenticate, store credentials securely, revoke/rotate keys
    - Demonstration:
        - claude login (browser-based OAuth) or export CLAUDE_API_KEY="..."
        - Verify: claude session start (or claude whoami)
        - Store keys: OS keychain / dotenv + .gitignore
        - Safety note: do not commit .env or keys
3. Unit 3 — Quickstart: Create first project session
    - Level: Beginner
    - Objective: Start a session, perform file inspection and simple edits
    - Demonstration:
        - claude session start --name quickstart
        - claude open src/main.py
        - claude edit src/main.py --message "Fix off-by-one bug" --dry-run
        - Inspect diff, then claude edit ... --apply (or commit via git)
        - Action: Confirm exact flags (--dry-run/--apply) in CLI version
4. Unit 4 — Project context: CLAUDE.md basics
    - Level: Beginner
    - Objective: Author CLAUDE.md to guide assistant behavior
    - Demonstration:
        - Add CLAUDE.md with purpose, build/run/test commands, important files, do-not-edit list
        - Show claude session reading CLAUDE.md context (simulate using the assistant)
        - Provide template (see Appendix)

5. Unit 5 — Core CLI commands (open, edit, run, search, commit)

    - Level: Beginner
    - Objective: Learn safe workflows for core commands
    - Demonstration:

        - claude search "TODO" --glob "src/**"
        - claude open src/utils.py --line 120
        - claude edit src/utils.py --message "Replace regexp with simple parser" --dry-run
        - Review diff, then claude commit --message "Refactor parser"
        - Safety: Always preview edits; avoid auto-apply on main branches

6. Unit 6 — VS Code integration (install extension, workflow)

    - Level: Beginner
    - Objective: Set up and use VS Code extension
    - Demonstration:

        - Install extension from Marketplace
        - Link auth token (via extension or sync)
        - Start session in VS Code, accept suggestions, and sync with CLI session
        - Action: Validate extension feature parity and exact flows

7. Unit 7 — Simple code generation and test creation

    - Level: Beginner
    - Objective: Generate a helper function and unit tests
    - Demonstration:

        - Ask Claude to add helper parseTimestamp and tests
        - Run tests locally (claude run "npm test" or native test command)
        - Iterate until tests pass

8. Unit 8 — Basic safety and workspace hygiene

    - Level: Beginner
    - Objective: Teach preview/dry-run, .gitignore, and secret hygiene
    - Demonstration:

        - Reject an edit that references .env content
        - Add patterns to do-not-edit section in CLAUDE.md
        - Configure repo to prevent auto-apply in protected branches

Intermediate (Units 9–18) 9. Unit 9 — CLI configuration and advanced setup

- Level: Intermediate
- Objective: Project-level config, default model & timeout settings
- Demonstration:

    - Create .clauderc or claude config set default_model=claude-code-... timeout=60
    - Override model in a session: claude session start --model
    - Action: Provide .clauderc example schema for students

1. Unit 10 — Sessions, workspaces, and context management

- Level: Intermediate
- Objective: Named sessions, persistence, and switching
- Demonstration:
  - claude session start --name feature-A
  - claude session list
  - claude session switch feature-B
  - Show the influence of session history on responses

2. Unit 11 — Model selection and switching strategy
   - Level: Intermediate
   - Objective: Tradeoffs between latency/cost and capability; when to switch
   - Demonstration:
     - Run same prompt on smaller model vs large model and compare diffs for a complicated refactor
     - Heuristic rules: larger model for multi-file reasoning, smaller for formatting
3. Unit 12 — Prompting structure and instruction layering
   - Level: Intermediate
   - Objective: Build layered instructions (system/task/constraints)
   - Demonstration:
     - Compose system prompt (do not edit tests), task prompt (what to change), constraints (no breaking API)
     - Measure iterations and reduced back-and-forth
4. Unit 13 — Command scripting and repeatable workflows
   - Level: Intermediate
   - Objective: Automate chains of commands (scripts)
   - Demonstration:
     - Create shell script: run tests, if fail call claude run to propose fix, claude edit afterwards, rerun tests
     - Emphasize idempotency and safe retries
5. Unit 14 — Debugging workflows and iterative refinement
   - Level: Intermediate
   - Objective: Reproduce test failures, analyze stack traces, and fix in iterations
   - Demonstration:
     - Reproduce failing test, use claude open to inspect implementations, propose edits, run tests again
6. Unit 15 — Multi-file reasoning and refactoring
   - Level: Intermediate
   - Objective: Chunked and staged refactors across files with human approval
   - Demonstration:
     - Refactor a class from monolith to smaller modules, commit incremental changes, run tests between steps

7. Unit 16 — File awareness, repository search, and codebase mapping
    - Level: Intermediate
    - Objective: Summarize repo, index key files, identify hotspots
    - Demonstration:
        - claude search "authentication" --glob "packages/**"
        - Generate short architecture summary and dependency diagram
8. Unit 17 — Tool usage and function calling behavior
    - Level: Intermediate
    - Objective: Understand command-as-function semantics and when Claude will call commands
    - Demonstration:
        - Use claude run to execute tests and then have Claude propose edits based on run output
9. Unit 18 — Git workflows: automated commits, PRs, and code review assistance
    - Level: Intermediate
    - Objective: Integrate Claude into branching/PR workflows
    - Demonstration:
        - Create branch, claude edit applied to branch, claude pr --title "Fix X" --body "..." to draft PR
        - Claude generates PR description, changelog, and review checklist

Advanced (Units 19–36) 19. Unit 19 — Agent orchestration patterns - Level: Advanced - Objective: Multi-step autonomous flows with verification and human-in-loop checkpoints - Demonstration: - Script an automated loop: run tests → fix errors → lint → human approval for apply step - Emphasize safety toggles

1. Unit 20 — Building custom tools and connectors
    - Level: Advanced
    - Objective: Extend CLI with custom plugins, internal API connectors
    - Demonstration:
        - Register a custom "internal-deploy" tool or wrapper script and show how Claude can invoke it (via run or function-call API)
2. Unit 21 — Function calling and tool sandboxing
    - Level: Advanced
    - Objective: Secure remote calls, sandboxing shell commands, and rollback procedures
    - Demonstration:
        - Execute potentially hazardous script inside a disposable container (Docker) and demonstrate rollback/revert pattern
3. Unit 22 — Large codebase navigation and scaling strategies
    - Level: Advanced

- Objective: Partitioning, indexing, and per-package CLAUDE.md to reduce context
- Demonstration:
    - Build per-package manifests, precompute file summaries, and show improved assistant performance

4. Unit 23 — Memory, context window management, and persistent knowledge
    - Level: Advanced
    - Objective: Long-running project strategies: memory usage, pruning, and design doc injection
    - Demonstration:
        - Keep project decisions preserved across sessions via explicit memory store or exported session summaries
5. Unit 24 — Performance tuning and limits mitigation
    - Level: Advanced
    - Objective: Reduce latency and cost, optimize token usage
    - Demonstration:
        - Profile a scenario, refactor prompts into concise instructions, batch edits to reduce round trips
6. Unit 25 — Safety, governance, and enterprise guardrails
    - Level: Advanced
    - Objective: Role-based permissions, commit gating, secret protection
    - Demonstration:
        - Configure repo-level safety settings (policy file), demonstrate rejection of edits touching secrets
7. Unit 26 — Testing, CI/CD, and reproducibility with Claude Code
    - Level: Advanced
    - Objective: Integrate Claude into CI to produce suggestions and machine-readable reports
    - Demonstration:
        - CI step that runs tests and posts Claude-generated suggestions as PR comments or issues
8. Unit 27 — Autonomous agents vs human-in-loop patterns
    - Level: Advanced
    - Objective: Compare and design workflows that choose between fully autonomous vs review gates
    - Demonstration:
        - Build two-mode pipeline (suggest vs apply) with audit logs
9. Unit 28 — Debugging at scale and observability
    - Level: Advanced

- Objective: Track actions, build telemetry for assistant-driven changes
- Demonstration:
  - Action logs and dashboard showing edits, session IDs, approver, and diff summaries

10. Unit 29 — Complex refactors, migrations, and API changes
    - Level: Advanced
    - Objective: Multi-repo or monorepo migrations with validation and staged rollout
    - Demonstration:
      - Migrate API v1 → v2 across modules, generate migration scripts and tests, stage rollout via feature flags
11. Unit 30 — Benchmarking Claude Code vs other tools
    - Level: Advanced
    - Objective: Define metrics and execute empirical comparisons
    - Demonstration:
      - Run standardized tasks (bug fixes, refactors), measure time-to-fix, accuracy, and cost; produce a results report
12. Unit 31 — Custom prompt libraries and instruction templates
    - Level: Advanced
    - Objective: Centralize prompt templates for teams
    - Demonstration:
      - Create a prompt library (refactor-template) and apply it across repos programmatically
13. Unit 32 — Security review and vulnerability remediation workflows
    - Level: Advanced
    - Objective: Find and remediate security issues via assistant guidance
    - Demonstration:
      - Run dependency checks, ask Claude for patch suggestions, apply and run tests
14. Unit 33 — Enterprise deployment, policy, and governance
    - Level: Advanced
    - Objective: Prepare SSO/managed accounts, policy mapping, and auditing
    - Demonstration:
      - Draft enterprise policy docs, map roles, and show integration steps (action: validate against Anthropic enterprise docs)
15. Unit 34 — Creating teaching exercises and lab infrastructure
    - Level: Advanced
    - Objective: Build sandboxed labs, autograder integration, and grading rubrics
    - Demonstration:

        - Create lab repo with hidden tests and an autograder; show instructor view vs student view

16. Unit 35 — Real-world capstone: Full-stack feature build with Claude Code
    - Level: Advanced
    - Objective: End-to-end design, implement, test, PR, and deploy a feature
    - Demonstration:
        - Implement feature (backend endpoint + frontend UI) with tests, PR, changelog, and CI verification
17. Unit 36 — Course wrap-up, evaluation, and continuous improvement
    - Level: Advanced
    - Objective: Present capstones, grade, and iterate on course materials
    - Demonstration:
        - Student presentations, instructor reviews, rubric-based scoring and feedback loop

Setup workflow validation (step-by-step)

- Supported platforms:
    - macOS, Linux, Windows (WSL recommended for Windows developers)
    - Minimum tools: Git, optional Node/npm (if using npm installer), Docker (optional, recommended for sandboxing)
- Installation methods (validated and recommended patterns):
    - npm (if distributed via npm): npm i -g @anthropic/claude-code
    - Homebrew (macOS): brew install claude-code (confirm upstream formula)
    - Native installers: platform-specific packages (confirm availability)
    - VS Code: install extension from Marketplace; ensure extension version matches CLI
    - Post-install check: claude --version
    - Action: Validate actual installer command and package name at publish time; include checksums for native installers if provided
- Authentication & key management:
    - Flow 1: claude login (browser OAuth) — recommended for interactive users
    - Flow 2: environment variable CLAUDE_API_KEY="..." for CI or automated runners
    - Secure storage:
        - macOS Keychain / Windows Credential Manager / Linux secret store (recommended where supported)
        - .env + .gitignore for local dev only; avoid committing keys
    - Best practices: rotate keys, per-project keys, least privilege tokens, set expiry
- Project initialization:
    - claude session start --name or claude init (if CLI provides)

    - Add CLAUDE.md with schema fields:
        - purpose
        - build: steps
        - test: commands
        - important_files: list
        - do_not_edit: patterns
        - editor_prefs: (indent, line-length)
        - example content included in Appendix
- Environment & configuration:
    - claude config set default_model=
    - claude config set timeout=
    - Project-level .clauderc placed in repo root (JSON/YAML). Example schema recommended in Appendix.
    - VS Code extension: configure token sync or local token file and map workspace to session
- Sandboxing and safety:
    - Use Docker for executing untrusted code: claude run in containerized environment
    - For CI, run Claudia suggestions in ephemeral environments before applying
- Troubleshooting & diagnostics:
    - CLI debug: claude --verbose or claude --log
    - Network/auth errors: check proxy, token expiry
    - Model unavailability: switch to fallback model or retry with backoff
    - PATH issues: ensure installer writes to location in PATH and shell restart
    - Action: Include a troubleshooting appendix with sample log outputs and resolution steps

Command documentation validation

- Command system overview (validated concepts)
    - Categories:
        - Session/control: session start/list/switch/export
        - Inspection: open, search
        - Execution: run, test
        - Modification: edit, apply, commit, stash/undo
        - Repo integration: commit, pr
        - Utilities: config, login, version
    - Patterns:
        - Dry-run/preview first
        - Command outputs are structured (plain text + JSON/log option) for CI integration
        - Function-call model: assistant may choose to call commands or propose edits depending on permissions

- Per-command reference (canonical examples; confirm flags in live CLI):
  - claude session start [--name] [--model] — starts/attaches to a named session
    - Example: claude session start --name feature-123 --model claude-code-2
    - Safety: Starting a session does not apply changes
  - claude session list — lists active sessions
  - claude session switch — switch to named session
  - claude session export — export session history to file (JSON or text)
  - claude open [--line] — fetch file/specific lines into context
    - Example: claude open src/auth.py --line 120
  - claude edit --message "" [--dry-run | --apply] — propose or apply edits
    - Example: claude edit src/auth.py --message "Fix password hashing bug" --dry-run
    - Safety: Require explicit --apply to modify files
  - claude search "" [--glob ""] — semantic + text search
    - Example: claude search "rate limiter" --glob "services/**"
  - claude run "" [--capture] — execute command and capture stdout/stderr
    - Example: claude run "pytest tests/test_user.py" --capture
    - Security: Sanitize outputs before injecting to prompts
  - claude test [--unit | --integration | --filter] — run test sets (if integrated)
  - claude commit --message "" [--author ""] — create commit for CLAUDE-applied changes
    - Example: claude commit --message "Fix hashing bug"
  - claude pr --title "" --body "<body>" — draft PR and open in remote (or print PR body)</li> <li>claude stash / claude undo — revert last assistant-applied change (soft revert with audit trail)</li> <li>claude config set|get|unset <key> — manage configuration; supports project-level and global scope</li> <li>claude login / logout — authentication flow</li> <li>claude version — print installed version</li> </ul> </li> <li>Execution flow & semantics: <ul> <li>Commands translate to internal API calls; some commands return structured JSON (useful for CI)</li> <li>Two styles: "Propose then apply" vs "Autonomous apply" <ul> <li>Recommend default: propose & require human approval</li> <li>For automation lab: allow opt-in apply mode with policy checks</li> </ul> </li> </ul> </li> <li>Hooks & tools: <ul> <li>Pre-command hooks: run linters or tests before editing</li> <li>Post-command hooks: run CI or tests to validate edits</li> <li>Custom tool registration: documented extension points for integrating wrapped scripts/services</li> </ul> </li> <li>Error handling & idempotency: <ul> <li>Encourage idempotent scripts and clear retry semantics</li> <li>Use explicit conflict detection when branches diverge</li> <li>Provide guidance on safe rollbacks: claude stash / git revert</li> </ul> </li> </ul> Command coverage confirmation (auditor notes) <ul> <li>Coverage: The course covers all major categories and common

workflows.</li> <li>Gaps to resolve before publication: <ul> <li>Confirm exact flag names (--dry-run, --apply, --capture) against current CLI release.</li> <li>Provide sample JSON outputs for session export and run capture for CI examples.</li> <li>Define error exit codes for commands in instructor materials.</li> </ul> </li> </ul> Model and context explanation validation <ul> <li>Model selection & heuristics (validated guidance) <ul> <li>Tradeoffs: <ul> <li>Larger models: better for complex reasoning (multi-file refactor, generating tests, design decisions); higher cost & latency</li> <li>Smaller models: lower cost, faster, good for formatting and simple edits</li> </ul> </li> <li>How to set: <ul> <li>Global: claude config set default_model=<model></li> <li>Per-session: claude session start --model <model></li> <li>Per-command override: (if CLI supports) --model on edit/run</li> </ul> </li> <li>Heuristics recommended in course: <ul> <li>Use high-capacity models for test creation, cross-file refactors, migration planning</li> <li>Use smaller models for single-file linting, formatting, or content changes</li> </ul> </li> </ul> </li> <li>Context handling (validated architecture) <ul> <li>Sources: <ul> <li>CLAUDE.md (project manifest)</li> <li>On-demand file fetches (claude open)</li> <li>Session history</li> <li>Command outputs (claude run)</li> <li>External injected docs (design.md)</li> </ul> </li> <li>Context window & prioritization: <ul> <li>The assistant will have a finite context window (tokens) — teach strategies: <ul> <li>Scope file fetches narrowly (open specific files & ranges)</li> <li>Summarize large files before adding them</li> <li>Precompute indexes / embeddings if supported (discuss as advanced optimization)</li> </ul> </li> <li>Techniques to reduce context usage: <ul> <li>Provide explicit file lists in prompts</li> <li>Use CLAUDE.md to highlight critical artifacts</li> </ul> </li> </ul> </li> </ul> </li> <li>Memory & session behavior: <ul> <li>Session history persists by default (if supported) — includes prior edits and assistant reasoning</li> <li>Memory features (if present) can store high-value summaries (design decisions)</li> <li>Best practices: <ul> <li>Reset or archive sessions periodically</li> <li>Use named sessions for concurrent work</li> <li>Monitor for context bloat and prune or export session history</li> </ul> </li> </ul> </li> <li>Privacy & secrets: <ul> <li>Never put secrets in CLAUDE.md</li> <li>Sanitize logs and command outputs before storing or injecting into prompts</li> <li>Use least-privilege tokens for CI and automation agents</li> </ul> </li> </ul> Practical demonstration validation <ul> <li>Demo planning (validated) <ul> <li>Each unit includes a short lecture (15–20 min) + live demo (20–30 min) + lab (30–60 min)</li> <li>Demo scenarios (ready to use): <ul> <li>Demo 1 (Intro): Fix failing test — run tests, show failing output, claude open to inspect, claude edit to propose change, run tests again</li> <li>Demo 2 (Intermediate): Refactor auth module — open files, plan refactor, stage changes across files, run tests</li> <li>Demo 3 (Advanced): Automated migration — agent runs across multiple services, applies staged changes, runs integration tests</li> </ul> </li> <li>Lab infrastructure requirements: <ul> <li>Starter repos for each unit: minimal reproducible examples and hidden tests for grading</li> <li>Docker sandbox for running untrusted code from students</li> <li>CI pipelines (GitHub Actions / GitLab CI) to run autograder steps, and demonstration pipelines showing Claude automation hooks</li> </ul> </li> <li>Grading & assessment: <ul> <li>Rubrics: correctness (tests pass), commit hygiene (clear messages), safe edits (no secrets), prompt design quality (reusability)</li> <li>Autograder: validate tests, run

static checks, and optionally validate that student invoked Claude appropriately (via session export logs)</li> </ul> </li> </ul> Practical roadmap summary (course delivery & timelines) <ul> <li>Week 0: Instructor prep — setup lab infra, verify CLI/extension versions, seed repos</li> <li>Week 1 (Beginner): Units 1–8 <ul> <li>Goals: install, authenticate, CLAUDE.md, basic CLI, VS Code integration</li> <li>Labs: Hello Claude utility + tests</li> </ul> </li> <li>Week 2 (Intermediate): Units 9–18 <ul> <li>Goals: sessions, models, prompting, multi-file refactor, Git workflows</li> <li>Labs: Refactor Product Module; create PR with tests</li> </ul> </li> <li>Week 3 (Advanced): Units 19–36 (split across days/weeks as appropriate) <ul> <li>Goals: agents, custom tools, CI integration, governance, performance</li> <li>Labs: Automated Release Assistant; benchmarking tasks; security remediation lab</li> </ul> </li> <li>Final week: Capstone project (Unit 35) and presentations (Unit 36)</li> <li>Suggested pacing: 3 hours lecture + 2–4 hours lab per major topic day; flexible for multi-week cohorts</li> </ul> Capstone & assessment details <ul> <li>Capstone: Full-stack feature where Claude: <ul> <li>Summarizes existing code related to feature</li> <li>Generates missing tests and implementation</li> <li>Iteratively fixes failing tests until green</li> <li>Produces PR with changelog and release notes</li> <li>Sets up CI checks to validate gating</li> </ul> </li> <li>Evaluation criteria: <ul> <li>Correctness: tests pass, behavior matches spec</li> <li>Maintainability: clear commits and modular changes</li> <li>Use of assistant: proper session usage, prompt templates, and CLAUDE.md application</li> <li>Security & policy compliance: no secrets leaked, policy rules followed</li> </ul> </li> </ul> Comparison with other AI coding tools (concise) <ul> <li>Differentiators: <ul> <li>Terminal-first agentic model (explicit open/run/edit primitives)</li> <li>CLAUDE.md project manifest gives repo-level guidance</li> <li>Strong session/workspace model for multi-file reasoning</li> </ul> </li> <li>Typical alternatives and when to choose Claude Code: <ul> <li>GitHub Copilot: best for inline completions and editor-centric workflows</li> <li>Copilot Labs / Chat-based tools: conversational but less integrated with command lifecycle</li> <li>Tabnine / Codeium: completion-first assistants</li> <li>Choose Claude Code when: repository-wide refactors, test-driven iterative fixes, or terminal-centric agentic automation is needed</li> </ul> </li> </ul> Safety & guardrail recommendations (course content validated) <ul> <li>Default to preview/dry-run mode; require explicit apply and commit</li> <li>Enforce protected branches and role-based approvals in production repos</li> <li>Sanitize inputs (test logs, stack traces) before providing to assistant</li> <li>Use ephemeral environments (Docker) for executing assistant-proposed code</li> <li>Maintain audit logs: session ID, action diffs, approver, timestamp</li> <li>Add exercises highlighting pitfalls (e.g., secret exposure, incorrect API usage)</li> </ul> Command & content appendices (actionable artifacts) <ul> <li> Appendix A — CLAUDE.md minimal template (to ship with course) <ul> <li>Example (students copy into repo root): <ul> <li>purpose: "Service for user account management"</li> <li>build: "npm install && npm run build"</li> <li>test: "npm test"</li> <li>important_files: <ul> <li>src/auth/*</li> <li>src/services/user_service.py</li> </ul> </li> <li>do_not_edit: <ul> <li>.env</li> <li>secrets/*</li> </ul> </li> <li>editor_prefs: <ul> <li>indent: 2</li> <li>line_length: 100</li> </ul> </li> </ul> </li> </ul> </li> <li> Appendix B — Sample .clauderc (JSON) <ul> <li>{ "default_model": "claude-code-2", "timeout_seconds": 60,

"apply_mode_default": "preview", "protected_branches": ["main", "production"] }</li> <li>Action: Finalize schema names to match CLI implementation.</li> </ul> </li> <li> Appendix C — Sample shell script for command chaining (lab) <ul> <li>#!/usr/bin/env bash set -e npm test || { echo "Tests failed, asking Claude for suggestion..." claude run "pytest -q" --capture > last_test_output.txt claude edit src/module.py --message "Fix failing test based on output" --dry-run <h1>Review diff then:</h1> <h1>claude edit ... --apply</h1> <h1>npm test</h1> }</li> </ul> </li> </ul> Model & context explanation validation (concise checklist) <ul> <li>Confirm available model names and token limits from official docs</li> <li>Document heuristics: when to switch models; example thresholds</li> <li>Provide worked examples showing context truncation and mitigation strategies (summaries, file scoping)</li> <li>Include sample session export JSON to teach how to persist or audit reasoning</li> </ul> Course delivery readiness checklist (items verified / recommended) <ul> <li>Verified and complete: <ul> <li>Unit-level learning path and demonstrations</li> <li>Practical labs aligned with lectures</li> <li>Capstone and assessment rubric</li> <li>Safety & sandboxing guidance</li> </ul> </li> <li>Items to finalize before publishing: <ul> <li>Confirm exact CLI command flags, exit codes, and JSON response schemas against the installed CLI (replace placeholders if necessary)</li> <li>Produce starter repos for each unit (Beginner → Advanced)</li> <li>Provide autograder harness and sandbox Docker images</li> <li>Confirm enterprise SSO & policy integration steps with Anthropic (vendor docs)</li> <li>Create instructor slide decks and demo scripts (I can produce these on request)</li> </ul> </li> </ul> Final course validation statement (auditor integrity summary) <ul> <li>Structural validation: The course is logically structured, progressive, and covers the complete workflow from installation to advanced agentic orchestration. Units build appropriately from simple to complex topics and include measurable demonstrations.</li> <li>Module completeness confirmation: All required modules are present. Recommended minor additions: exact CLI reference appendix (match installed CLI), full .clauderc schema examples, and enterprise SSO steps.</li> <li>Setup workflow validation: Installation, authentication, project initialization, and sandboxing steps are complete and actionable; they include troubleshooting guidance. Action: confirm actual installer package names and flags at release time.</li> <li>Command documentation confirmation: Core commands and categories are covered and mapped to lab activities. Action: verify command flag names and ensure sample outputs (JSON/log) are captured for CI examples.</li> <li>Model & context explanation verification: Guidance is accurate for model selection tradeoffs, context management, and session behavior. Include exact token limits and model names from official docs in final materials.</li> <li>Practical demonstration validation: Demos and labs are feasible and reflect real-world developer workflows. Infrastructure requirements (Docker, CI) are identified and planned.</li> <li>Final integrity statement: With two final validations (confirm CLI/extension syntaxes and add production starter repos + autograder), this course blueprint is ready for publication. It provides a production-aligned, safe, and practical learning pathway for developers to adopt Claude Code in professional workflows.</li> </ul> Actionable next steps (recommended) <ul> <li>Validate CLI & extension version and replace placeholder flags with exact flags from installed version</li> <li>Generate starter lab repositories and hidden tests for automated grading</li> <li>Produce instructor slide decks, demo scripts, and student lab guides (I can

generate these)</li> <li>Build Docker sandbox images and CI job templates for labs</li> <li>Create a one-page cheat sheet (commands & prompt templates) for students</li> </ul> If you'd like, I can now: <ul> <li>Generate ready-to-use lab repos and step-by-step exercise guides for each unit</li> <li>Produce instructor slide decks and demo scripts</li> <li>Create a student assessment rubric and autograder integration plan</li> </ul> Which deliverable should I produce next (sample lab, CLAUDE.md template, or instructor slide deck)?